

LAPORAN PRAKTIKUM

PENGOLAHAN CITRA

SEGMENTASI WARNA MERAH PADA CITRA MOBIL

MENGGUNAKAN OPENCV DI RASPBERRY PI

Disusun oleh:

Ahmad Hanif Abiyyu Khrisna

NRP: 2125640006

PROGRAM STUDI D4 TEKNIK ELEKTRONIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

2026

BAB I TEORI DASAR

1.1 Segmentasi Citra

Segmentasi citra adalah proses membagi sebuah citra digital menjadi beberapa wilayah (region) yang memiliki kesamaan karakteristik tertentu, seperti intensitas, warna, atau tekstur (Gonzalez & Woods, 2018). Tujuannya adalah menyederhanakan representasi citra agar lebih mudah dianalisis pada tahap selanjutnya, seperti pengenalan objek, deteksi tepi, atau klasifikasi. Dalam konteks praktikum ini, segmentasi dilakukan berbasis warna untuk memisahkan piksel objek berwarna merah dari latar dan objek lain dalam citra.

1.2 Thresholding Warna

Thresholding warna adalah operasi titik (point operation) yang memisahkan piksel berdasarkan rentang nilai warnanya. Setiap piksel diuji terhadap batas bawah (lower) dan batas atas (upper) di tiap kanal warna. Apabila seluruh kanal piksel memenuhi syarat $L \leq p \leq U$, piksel tersebut diberi nilai 255 (putih, dianggap target), sedangkan piksel yang gagal diberi nilai 0 (hitam). Output operasi ini berupa citra biner (mask) yang menunjukkan lokasi piksel target. Pada OpenCV, operasi ini diimplementasikan oleh fungsi `cv2.inRange()`.

1.3 Ruang Warna RGB dan HSV

Ruang warna RGB merepresentasikan warna sebagai kombinasi tiga kanal: Red, Green, dan Blue. Model ini bersifat aditif dan cocok untuk perangkat tampilan (monitor, kamera). Namun untuk segmentasi warna, RGB kurang ideal karena perubahan kondisi pencahayaan menggeser ketiga kanal secara bersamaan, sehingga rentang threshold yang valid menjadi sulit ditentukan.

Ruang warna HSV (Hue, Saturation, Value) adalah representasi silindris dari RGB yang lebih mendekati persepsi manusia terhadap warna (OpenCV Documentation, 2024). Komponen HSV terdiri dari:

- Hue (H): jenis warna, dinyatakan sebagai sudut pada lingkaran warna. Pada OpenCV nilainya berkisar 0–179 (bukan 0–359) karena disimpan dalam 8-bit unsigned integer.
- Saturation (S): kemurnian warna, berkisar 0–255. Nilai rendah mendekati abu-abu, tinggi mendekati warna murni.
- Value (V): kecerahan, berkisar 0–255. Nilai rendah mendekati hitam, tinggi mendekati warna terang.

Karena Hue terisolasi dari informasi kecerahan (V), segmentasi warna di ruang HSV jauh lebih tahan terhadap variasi pencahayaan dibandingkan RGB.

1.4 Karakteristik Khusus Warna Merah pada HSV

Hue bersifat siklis: nilai 0° dan 360° merepresentasikan warna yang sama (merah). Akibatnya, warna merah secara alami berada di kedua ujung skala Hue. Pada OpenCV, hal ini muncul sebagai dua rentang terpisah, yaitu sekitar Hue 0–10 dan Hue 170–180 (GeeksforGeeks, 2024). Untuk mendeteksi seluruh spektrum merah, perlu dibuat dua mask hasil `cv2.inRange()`, kemudian digabungkan menggunakan operasi bitwise OR.

BAB II METODOLOGI

2.1 Alat dan Bahan

- Hardware: Raspberry Pi 4B 8GB, microSD 32GB, adaptor USB-C 5V/3A, laptop Lenovo Legion (host SSH dan transfer file).
- Software: Raspberry Pi OS 13 (Trixie, kernel 6.12.75, aarch64), Python 3.11, OpenCV 4.10.0, NumPy 2.2.4.
- Citra dataset utama: foto komposit empat mobil Toyota (C-HR merah belakang/depan, C-HR putih, C-HR kuning, Yaris merah) dengan resolusi 800×450 px.
- Citra dataset tambahan: foto buah apel (apple.jpg) dan foto strawberry pada tanaman dengan latar daun hijau (strawberry.jpg).
- Akses RPi: SSH dari laptop ke RPi melalui jaringan WiFi yang sama, dengan Raspberry Pi Connect sebagai backup remote shell via browser.

2.2 Diagram Alir Pengolahan

Pipeline pengolahan citra disusun sebagai berikut:

```
Citra BGR (input)
↓
cv2.cvtColor (BGR → HSV)
↓
cv2.inRange (Hue 0–10, S 120–255, V 70–255) → mask1
cv2.inRange (Hue 170–180, S 120–255, V 70–255) → mask2
↓
cv2.bitwise_or (mask1, mask2) → mask gabungan
↓
cv2.morphologyEx (MORPH_OPEN, kernel 5×5)
cv2.morphologyEx (MORPH_CLOSE, kernel 5×5)
↓
cv2.bitwise_and (citra asli, citra asli, mask) → hasil segmentasi
```

2.3 Penjelasan Fungsi-Fungsi Utama

- a. `cv2.cvtColor(src, code)`: mengubah representasi piksel dari satu ruang warna ke ruang warna lain. Untuk konversi BGR ke HSV digunakan kode `cv2.COLOR_BGR2HSV`. OpenCV membaca file citra dengan urutan kanal BGR (bukan RGB), sehingga kode konversi yang benar adalah BGR2HSV.
- b. `cv2.inRange(src, lowerb, upperb)`: melakukan thresholding per-piksel-per-kanal. Piksel yang seluruh kanalnya berada di antara `lowerb` dan `upperb` akan diberi nilai 255, sisanya 0. Output berupa citra biner `uint8`.
- c. `cv2.bitwise_or(src1, src2)`: operasi OR per-bit antara dua mask. Digunakan untuk menggabungkan dua rentang Hue merah (0–10 dan 170–180) menjadi satu mask utuh.
- d. `cv2.bitwise_and(src1, src2, mask)`: operasi AND per-bit. Pola standar `bitwise_and(img, img, mask=mask)` dipakai untuk menampilkan citra asli hanya pada piksel yang aktif (255) di mask.
- e. `cv2.morphologyEx(src, op, kernel)`: melakukan operasi morfologi seperti OPEN (erosi diikuti dilasi) untuk menghilangkan bintik kecil, atau CLOSE (dilasi diikuti erosi) untuk menutup lubang dalam objek.

2.4 Parameter Threshold yang Diuji

Untuk memenuhi syarat eksperimen wajib (minimal 3 variasi threshold), tiga skenario disiapkan dengan parameter berikut:

Skenario	Lower H1/S/V	Upper H1	Lower H2/S/V	Upper H2
Default	(0, 120, 70)	(10, 255, 255)	(170, 120, 70)	(180, 255, 255)
Tight	(0, 200, 100)	(8, 255, 255)	(172, 200, 100)	(180, 255, 255)
Loose	(0, 60, 40)	(15, 255, 255)	(160, 60, 40)	(180, 255, 255)

Tabel 2.1 Variasi parameter threshold HSV pada tiga skenario eksperimen

2.5 Simulasi Variasi Pencahayaan

- Kondisi terang: V dikalikan faktor 1.6 ($V \times 1.6$, di-clip ke maksimum 255).
- Kondisi redup: V dikalikan faktor 0.4 ($V \times 0.4$).

Simulasi dilakukan pada kanal V di ruang HSV sehingga warna (Hue) tetap dipertahankan, hanya kecerahan yang berubah.

BAB III HASIL PADA DATASET UTAMA (MOBIL)

3.1 Citra Input

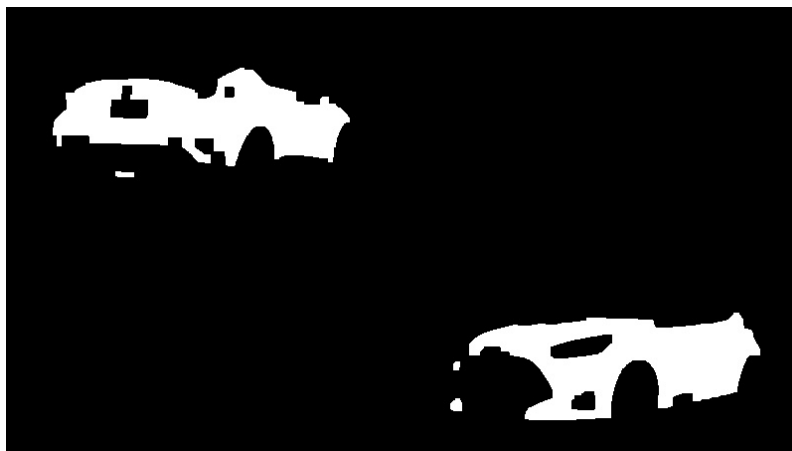
Citra input merupakan komposit empat mobil Toyota: dua C-HR merah (tampak belakang dan tampak depan), satu C-HR putih, satu C-HR kuning, dan satu Yaris merah.



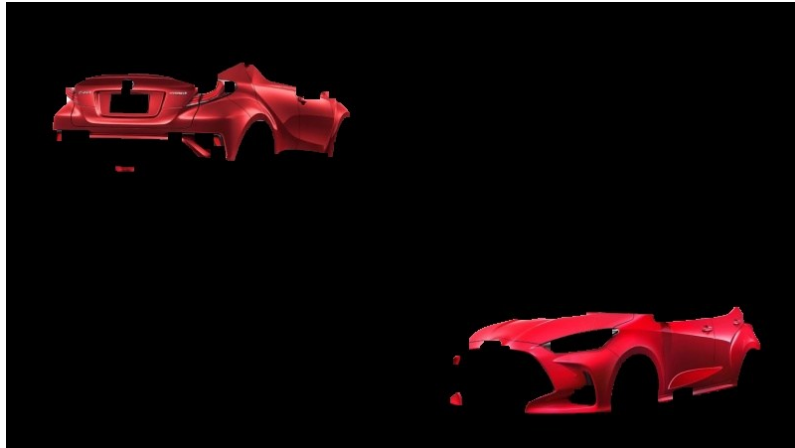
Gambar 3.1 Citra input dataset utama (mobil.jpg)

3.2 Hasil Skenario Default

Pada skenario default, algoritma berhasil mendeteksi body kedua mobil merah secara komprehensif. Mobil putih dan kuning tidak terdeteksi, sesuai harapan.



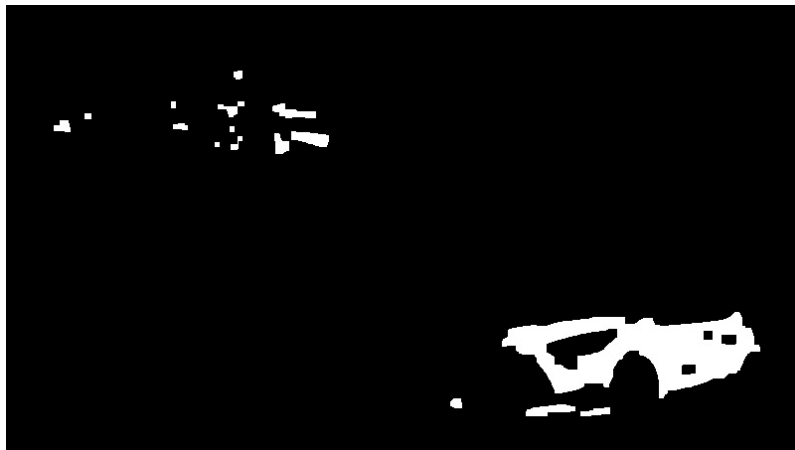
Gambar 3.2 Mask biner skenario default — area putih = piksel merah teridentifikasi



Gambar 3.3 Hasil segmentasi default

3.3 Hasil Skenario Tight

Pada skenario tight, hanya piksel merah yang sangat jenuh dan terang yang lolos. Body mobil tampil pecah-pecah dengan banyak lubang. Rasio piksel turun drastis dari 9.16% menjadi 3.77%.



Gambar 3.4 Mask skenario tight — false negative tinggi

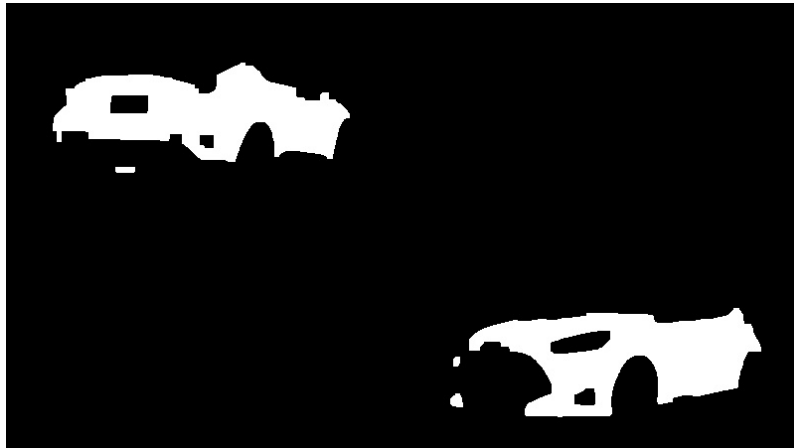
3.4 Hasil Skenario Loose

Pada skenario loose, area merah terdeteksi lebih utuh, namun bayangan dan piksel kemerahan di latar ikut lolos. Rasio piksel naik menjadi 10.26%.



Gambar 3.5 Mask skenario loose — false positive tinggi

3.5 Pengaruh Pencahayaan



Gambar 3.6 Mask kondisi terang ($V \times 1.6$)



Gambar 3.7 Mask kondisi redup ($V \times 0.4$) — body hampir hilang

3.6 Tabel Kuantitatif Mobil

Skenario	Red Pixel Ratio	Karakteristik
Default	9.16%	Body utuh, baseline optimal
Tight	3.77%	Body pecah, false negative tinggi
Loose	10.26%	Over-detection, false positive
Bright ($V \times 1.6$)	9.55%	Body solid, deteksi naik tipis
Dim ($V \times 0.4$)	3.58%	Body hampir hilang

Tabel 3.1 Rasio piksel merah pada lima skenario — dataset mobil

BAB IV EKSPERIMEN PADA DATASET TAMBAHAN

Sebagai pemenuhan requirement minimal 1 gambar tambahan, eksperimen diperluas ke dua dataset baru: foto apel (multi-warna pada satu frame) dan foto strawberry pada tanaman (objek merah di latar daun hijau). Tujuannya adalah menguji generalisasi algoritma di luar domain mobil.

4.1 Eksperimen pada Citra Apel

Citra apel berisi dua jenis apel (kuning-hijau di kiri dan merah di kanan) yang ditata dalam keranjang pasar. Tantangannya: apakah algoritma mampu memisahkan apel merah dari apel berwarna lain pada satu frame yang sama, dan menguji generalisasi di luar domain mobil.



Gambar 4.1 Citra input apel — dua jenis apel berwarna kontras



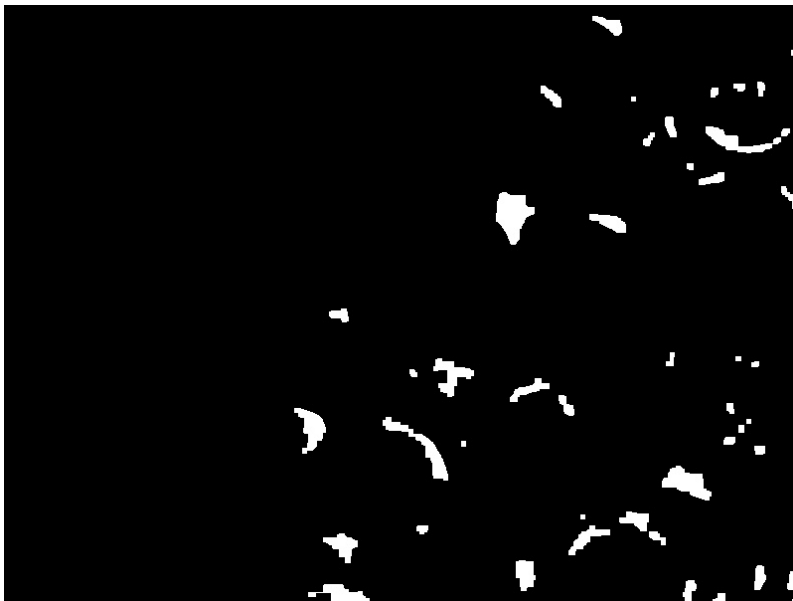
Gambar 4.2 Mask default pada apel — apel kuning tertolak sepenuhnya



Gambar 4.3 Hasil segmentasi default pada apel

Hasil pengamatan: cluster apel merah berhasil ter-segmentasi dengan rasio 41.88% — angka tinggi karena memang area merah sangat dominan di citra (>40% frame). Apel kuning di kiri tertolak total, membuktikan selektivitas algoritma. Beberapa lubang kecil di mask muncul akibat highlight (refleksi cahaya) pada permukaan apel yang membuat saturation lokal turun di bawah threshold.

4.2 Variasi Threshold pada Apel



Gambar 4.4 Mask tight pada apel — hanya area paling jenuh yang lolos (3.60%)



Gambar 4.5 Mask loose pada apel — sebagian apel kuning tepi mulai lolos (54.69%)

Pada skenario tight, rasio anjlok ke 3.60% — hanya area paling jenuh dan terang yang lolos, sebagian besar apel jadi hitam. Pada skenario loose, rasio melonjak ke 54.69% karena bayangan, area kemerahan tepi keranjang kayu, dan beberapa highlight di apel kuning ikut lolos.

4.3 Pengaruh Pencahayaan pada Apel



Gambar 4.6 Mask bright ($V \times 1.6$) — rasio 45.37%



Gambar 4.7 Mask dim ($V \times 0.4$) — rasio anjlok ke 7.16%

4.4 Eksperimen pada Citra Strawberry

Citra strawberry diambil dari tanaman strawberry dengan latar daun hijau. Citra ini sengaja dipilih karena merepresentasikan kondisi outdoor riil dengan tantangan: (a) objek merah target (strawberry matang), (b) objek non-target berwarna mirip namun beda saturasi (strawberry mentah putih-kekuningan), dan (c) latar yang dominan hijau.



Gambar 4.8 Citra input strawberry — kondisi outdoor dengan latar hijau



Gambar 4.9 Mask default pada strawberry — daun hijau tertolak total (11.85%)



Gambar 4.10 Hasil segmentasi pada strawberry

Hasil pengamatan strawberry: tiga strawberry matang (yang merah penuh) berhasil ter-segmentasi clean, sementara strawberry mentah (warna putih-kekuningan) dan seluruh latar daun hijau tertolak total. Ini bukti kuat selektivitas algoritma karena merah dan hijau berada di sisi berlawanan dalam roda warna Hue.

4.5 Variasi Threshold pada Strawberry



Gambar 4.11 Mask tight pada strawberry — area pusat saja yang lolos (7.96%)



Gambar 4.12 Mask loose pada strawberry — beberapa noise area mulai lolos (12.81%)

4.6 Pengaruh Pencahayaan pada Strawberry



Gambar 4.13 Mask bright ($V \times 1.6$) — rasio 12.00%



Gambar 4.14 Mask dim ($V \times 0.4$) — rasio 6.86%

4.7 Tabel Komparasi Multi-Dataset

Skenario	Mobil	Apel	Strawberry
Default	9.16%	41.88%	11.85%
Tight	3.77%	3.60%	7.96%
Loose	10.26%	54.69%	12.81%
Bright	9.55%	45.37%	12.00%
Dim	3.58%	7.16%	6.86%

Tabel 4.1 Rasio piksel merah pada tiga dataset (data otentik dari run di Raspberry Pi)

BAB V ANALISIS HASIL EKSPERIMEN

5.1 Analisis Trade-off Threshold

Dari tiga variasi threshold pada masing-masing dataset, terlihat pola yang konsisten:

- Menaikkan threshold (Tight) memperkecil false positive tetapi menaikkan false negative.
- Menurunkan threshold (Loose) memperkecil false negative tetapi memperbesar false positive.
- Default memberikan kompromi terbaik untuk masing-masing dataset.

Lonjakan paling ekstrem terjadi pada citra apel: dari Tight 3.60% naik ke Loose 54.69% — selisih 51%. Ini disebabkan komposisi warna citra apel yang hampir sebagian besar berwarna merah/kemerahan, sehingga sedikit pelonggaran S/V langsung menambah area lolos secara signifikan.

Pada strawberry, perubahan lebih moderat (7.96% → 12.81%) karena objek merah hanya menempati sebagian kecil frame (sekitar 12% area), dan latar daun hijau secara fundamental berbeda Hue-nya, sehingga pelonggaran threshold tidak banyak menambah false positive.

5.2 Analisis Pengaruh Pencahayaan

Penurunan V ($\text{Dim}, V \times 0.4$) memberikan dampak signifikan di semua dataset:

- Mobil: 9.16% turun ke 3.58% (-5.58%)
- Apel: 41.88% turun ke 7.16% (-34.72%)
- Strawberry: 11.85% turun ke 6.86% (-4.99%)

Apel paling parah karena banyak piksel awalnya berada di rentang V sedang (60–120) yang setelah dikalikan 0.4 menjadi di bawah ambang $V_{\min}=70$. Ini konsisten dengan sifat thresholding warna HSV: parameter $V_{\min}$ adalah variabel paling sensitif terhadap kondisi pencahayaan.

Pencahayaan terang (Bright, $V \times 1.6$) hanya menggeser rasio sedikit naik (mobil +0.39%, apel +3.49%, strawberry +0.15%). Ini karena V dibatasi di atas oleh $V_{\max}=255$ — piksel yang sudah jenuh tetap masuk rentang.

Mitigasi praktis yang direkomendasikan adalah menerapkan histogram equalization (`cv2.equalizeHist`) atau CLAHE pada kanal V sebelum thresholding, untuk meratakan distribusi V tanpa merusak Hue.

5.3 Analisis Generalisasi Lintas Domain

Eksperimen pada tiga dataset berbeda (mobil, apel, strawberry) membuktikan bahwa:

1. Algoritma HSV dual-range Hue tidak terikat domain — bekerja pada objek apa saja yang berwarna merah.
2. Selektivitas warna terbukti kuat: pada apel mobil putih dan kuning tertolak; pada strawberry latar hijau dan strawberry mentah tertolak.
3. Rasio piksel merah bukan ukuran akurasi — angka tinggi (apel 41.88%) bisa berarti deteksi sukses karena memang banyak objek merah, bukan over-detection.
4. Perlu konteks per-citra untuk interpretasi: rasio yang sama bisa bermakna berbeda tergantung komposisi citra.

BAB VI TANTANGAN NYATA DAN STRATEGI PERBAIKAN

6.1 Mengapa Benda Non-Target Ikut Terdeteksi?

Thresholding warna adalah operasi titik (point operation) yang hanya mempertimbangkan nilai HSV per piksel, tanpa memahami bentuk geometris atau konteks objek. Konsekuensinya, semua benda berwarna merah dengan rentang HSV yang sama akan ikut lolos mask, antara lain:

- Rambu lalu lintas merah (rambu larangan, stop sign).
- Lampu rem (brake light) mobil lain di jalan.
- Pakaian atau helm pejalan kaki/pengendara berwarna merah.
- Material bata, atap genteng kemerahan, atau bunga merah di pinggir jalan.
- Refleksi merah pada kaca atau permukaan logam mengkilap.
- Awan saat sunset yang memiliki tone kemerahan.

6.2 Strategi Perbaikan (Tanpa Deep Learning)

Beberapa pendekatan klasik dapat diterapkan untuk meminimalkan false positive:

- a. Filter geometri pada kontur. Setelah memperoleh mask, gunakan `cv2.findContours()` untuk mendapatkan daftar kontur, lalu buang kontur dengan luas terlalu kecil/besar (`cv2.contourArea`), aspect ratio bounding box tidak sesuai (`cv2.boundingRect`), dan solidity rendah (rasio area / convex hull area).
- b. Kombinasi multi-color space. Gabungkan threshold HSV dengan kanal a^* dari ruang warna Lab. Kanal a^* sangat sensitif terhadap sumbu merah-hijau, sehingga piksel valid hanya yang lolos di kedua ruang warna sekaligus.
- c. Region of Interest (ROI). Untuk aplikasi dengan kamera tetap (mis. CCTV pintu garasi), batasi pencarian pada ROI tertentu, abaikan area langit/jalan.
- d. Background subtraction (untuk video). Gunakan `cv2.createBackgroundSubtractorMOG2()` untuk memisahkan objek bergerak dari latar statis sebelum thresholding warna.
- e. White balance / color constancy. Algoritma Gray-World atau Retinex untuk menormalkan pencahayaan sebelum konversi HSV.
- f. Validasi bentuk dengan Hough Transform. `cv2.HoughCircles()` untuk mendeteksi roda di dalam blob merah; bila ditemukan dua lingkaran roda dengan jarak masuk akal, klasifikasikan sebagai kandidat mobil.
- g. Histogram equalization atau CLAHE pada kanal V. Memitigasi sensitivitas terhadap pencahayaan redup tanpa merusak Hue.

BAB VII KESIMPULAN

Berdasarkan keseluruhan eksperimen yang dilakukan, dapat ditarik kesimpulan:

5. Segmentasi warna merah berbasis ruang warna HSV dengan dua rentang Hue (0–10 dan 170–180) berhasil mendeteksi target merah secara akurat pada tiga dataset berbeda: mobil Toyota merah (mobil putih dan kuning tertolak), apel merah (apel kuning-hijau tertolak), dan strawberry matang (strawberry mentah dan latar daun hijau tertolak).
6. Trade-off threshold konsisten di semua dataset: Tight menurunkan rasio (false negative), Loose menaikkan rasio (false positive). Default memberikan kompromi terbaik.
7. Kondisi pencahayaan redup ($V \times 0.4$) berdampak paling signifikan, dengan penurunan rasio terbesar pada apel (-34.72%) karena banyak piksel jatuh di bawah ambang $V_{\min}=70$.
8. Algoritma terbukti mampu generalize lintas domain (mobil, apel, strawberry) tanpa modifikasi parameter, membuktikan bahwa pendekatan klasik HSV dual-range Hue masih relevan untuk aplikasi embedded vision.
9. Keterbatasan utama adalah ketidakmampuan membedakan benda lain berwarna merah serupa. Mitigasi dapat dilakukan dengan filter kontur geometris, kombinasi multi-color space, white balance, validasi bentuk dengan Hough Transform, atau CLAHE — semuanya tetap dalam koridor pengolahan citra klasik tanpa deep learning.

DAFTAR PUSTAKA

Bradski, G., & Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.

GeeksforGeeks. (2024). Choosing the correct upper and lower HSV boundaries for color detection with cv::inRange (OpenCV). Diakses dari <https://www.geeksforgeeks.org/computer-vision/choosing-the-correct-upper-and-lower-hsv-boundaries-for-color-detection-with-cv-inrange-opencv/>

Gonzalez, R. C., & Woods, R. E. (2018). Digital Image Processing (4th ed.). Pearson.

OpenCV Documentation. (2024). Color Spaces in OpenCV. Diakses dari <https://opencv.org/color-spaces-in-opencv/>

OpenCV Documentation. (2024). Changing Colorspaces — cv.cvtColor, cv.inRange. Diakses dari https://docs.opencv.org/4.x/df/d9d/tutorial_py_colorspaces.html

OpenCV Documentation. (2024). Morphological Transformations. Diakses dari https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

OpenCV Q&A Forum. Correct HSV InRange Values for 'Red' Objects. Diakses dari <https://answers.opencv.org/question/28899/correct-hsv-inrange-values-for-red-objects/>

Smith, A. R. (1978). Color gamut transform pairs. ACM SIGGRAPH Computer Graphics, 12(3), 12–19.

LAMPIRAN SOURCE CODE PROGRAM

File: segmentasi_merah.py

```
"""
Segmentasi Warna Merah pada Citra
Ahmad Hanif Abiyyu Khrisna
2125640006
STr-LJ Teknik Elektronika
Pengolahan Citra

Framework: Python 3 + OpenCV 4 + NumPy (no deep learning)
"""
import cv2
import numpy as np
import os
import argparse

# == PARAMETER DEFAULT (HSV ranges) =====
# Hue di OpenCV berskala 0-179 (bukan 0-359), karena disimpan dalam uint8.
# Warna merah "membungkus" titik 0, sehingga butuh DUA rentang terpisah.
LOWER_RED_1 = np.array([ 0, 120, 70]) # Rentang merah-bawah (0° - 10°)
UPPER_RED_1 = np.array([ 10, 255, 255])
LOWER_RED_2 = np.array([170, 120, 70]) # Rentang merah-atas (170° - 180°)
UPPER_RED_2 = np.array([180, 255, 255])

KERNEL = np.ones((5, 5), np.uint8) # Structuring element untuk morfologi

def segment_red(img_bgr,
                low1=LOWER_RED_1, up1=UPPER_RED_1,
                low2=LOWER_RED_2, up2=UPPER_RED_2,
                use_morph=True):
    """Segmentasi piksel merah dari citra BGR -> (mask biner, citra hasil)."""
    # 1. Konversi ruang warna BGR -> HSV
    hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)

    # 2. Threshold dua rentang Hue
    mask1 = cv2.inRange(hsv, low1, up1)
    mask2 = cv2.inRange(hsv, low2, up2)

    # 3. Gabungkan mask (OR logical) -> seluruh piksel merah jadi putih
    mask = cv2.bitwise_or(mask1, mask2)

    # 4. Bersihkan noise: OPEN hapus bintik kecil, CLOSE tutup lubang dlm objek
    if use_morph:
        mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, KERNEL, iterations=1)
        mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, KERNEL, iterations=2)

    # 5. Tampilkan hanya piksel merah dari citra asli
    result = cv2.bitwise_and(img_bgr, img_bgr, mask=mask)
    return mask, result

def adjust_brightness(img, factor):
    """Simulasi kondisi pencahayaan: factor>1 terang, factor<1 redup."""
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV).astype(np.float32)
    hsv[..., 2] = np.clip(hsv[..., 2] * factor, 0, 255)
    return cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2BGR)

def red_pixel_ratio(mask):
    return 100.0 * np.count_nonzero(mask) / mask.size

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--input', default='mobil.jpg')
    ap.add_argument('--outdir', default='output')
    ap.add_argument('--width', type=int, default=800,
```

```

        help='Resize width agar ringan di RPi (0 = no resize)')
args = ap.parse_args()

os.makedirs(args.outdir, exist_ok=True)

img = cv2.imread(args.input)
if img is None:
    raise FileNotFoundError(f'Tidak dapat membaca {args.input}')

if args.width > 0 and img.shape[1] > args.width:
    scale = args.width / img.shape[1]
    img = cv2.resize(img, (args.width, int(img.shape[0] * scale)),
        interpolation=cv2.INTER_AREA)

# ---- EKSPERIMEN 1: Threshold default -----
mask, res = segment_red(img)
cv2.imwrite(f'{args.outdir}/01_original.jpg', img)
cv2.imwrite(f'{args.outdir}/02_mask_default.jpg', mask)
cv2.imwrite(f'{args.outdir}/03_result_default.jpg', res)
print(f'[Default ] Red ratio = {red_pixel_ratio(mask):6.2f}%')

# ---- EKSPERIMEN 2: Threshold "ketat" (saturation tinggi) -----
mask_t, res_t = segment_red(
    img,
    low1=np.array([0, 200, 100]), up1=np.array([8, 255, 255]),
    low2=np.array([172, 200, 100]), up2=np.array([180, 255, 255]))
cv2.imwrite(f'{args.outdir}/04_mask_tight.jpg', mask_t)
cv2.imwrite(f'{args.outdir}/05_result_tight.jpg', res_t)
print(f'[Tight ] Red ratio = {red_pixel_ratio(mask_t):6.2f}%')

# ---- EKSPERIMEN 3: Threshold "longgar" -----
mask_l, res_l = segment_red(
    img,
    low1=np.array([0, 60, 40]), up1=np.array([15, 255, 255]),
    low2=np.array([160, 60, 40]), up2=np.array([180, 255, 255]))
cv2.imwrite(f'{args.outdir}/06_mask_loose.jpg', mask_l)
cv2.imwrite(f'{args.outdir}/07_result_loose.jpg', res_l)
print(f'[Loose ] Red ratio = {red_pixel_ratio(mask_l):6.2f}%')

# ---- EKSPERIMEN 4: Pencahayaannya terang & redup -----
bright = adjust_brightness(img, 1.6)
dim = adjust_brightness(img, 0.4)

mb, rb = segment_red(bright)
md, rd = segment_red(dim)
cv2.imwrite(f'{args.outdir}/08_bright_input.jpg', bright)
cv2.imwrite(f'{args.outdir}/09_bright_mask.jpg', mb)
cv2.imwrite(f'{args.outdir}/10_dim_input.jpg', dim)
cv2.imwrite(f'{args.outdir}/11_dim_mask.jpg', md)
print(f'[Bright ] Red ratio = {red_pixel_ratio(mb):6.2f}%')
print(f'[Dim ] Red ratio = {red_pixel_ratio(md):6.2f}%')

print(f'\n[OK] Semua hasil tersimpan di '{args.outdir}'')

if __name__ == '__main__':
    main()

```

LAMPIRAN VIDEO PRESENTASI

VIDEO YOUTUBE